WaveAlert

Backend Tesztelési Dokumentáció

Balatonmáriafürdői Vízisport Egyesület Tagnyilvántartó Rendszer

Verzió: 1.0.0 | 2026. április | Teszt eszköz: Postman | GitHub: github.com/ManyiKornel755/BMFVSE

1. Összefoglaló

Teszt Összefoglaló

Metrika	Érték
Összes teszt collection	4
Összes request	19
Authentikáció tesztek	4
Tag kezelés tesztek	5
Edzés tesztek	5
Felhasználó tesztek	5
Teszt eszköz	Postman
Base URL	http://localhost:5000

Collection Áttekintés

Collection	Endpoint csoport	Requestek száma	Módszerek
Authentication Tests	/api/auth	4	POST, GET
Members Tests	/api/members	5	GET, POST, PUT, DELETE
Trainings Tests	/api/trainings	5	GET
Users Tests	/api/users	5	GET, POST, PATCH, DELETE

2. Projekt Áttekintés

WaveAlert Backend API

A WaveAlert backend egy Node.js alapú RESTful API, amely a Balatonmáriafürdői Vízisport Egyesület tagnyilvántartó rendszerét szolgálja ki. Az API a következő fő modulokat tartalmazza:

- Authentikáció és jogosultságkezelés (JWT token alapú)
- Tagok nyilvántartása és kezelése
- Edzések és rendezvények kezelése
- Felhasználók adminisztrációja

Technológiai Stack

- Backend: Node.js + RESTful API
- Adatbázis: SQL Server
- Authentikáció: JWT (Bearer token)
- Tesztelő eszköz: Postman
- Base URL: http://localhost:5000

Environment Változók

Változó	Leírás	Értéke
{{base_url}}	API alap URL	http://localhost:5000
{{auth_token}}	JWT Bearer token (login után beállítva)	dinamikus
{{user_id}}	Bejelentkezett user ID-ja	dinamikus
{{test_member_id}}	Létrehozott teszt tag ID-ja	dinamikus
{{new_user_id}}	Létrehozott teszt user ID-ja	dinamikus
{{random_email}}	Egyedi email regisztrációhoz	dinamikus

3. Tesztelési Stratégia

Postman Collection Runner

A backend tesztek Postman Collection Runner segítségével futtathatók. A tesztek sorrendben futnak le, mivel egyes tesztek az előző requestek eredményére (pl. auth_token, test_member_id) támaszkodnak.

Teszt Típusok

Státuszkód ellenőrzések

Minden request ellenőrzi a visszakapott HTTP státuszkódot. Sikeres műveleteknél 200 (OK) vagy 201 (Created), hibáknál 401 (Unauthorized) vagy 404 (Not Found) a várt válasz.

Response struktúra ellenőrzések

A tesztek ellenőrzik, hogy a JSON response tartalmazza-e a kötelező mezőket (id, email, roles, stb.) és hogy a mezők a megfelelő típusúak-e.

Biztonsági ellenőrzések

Külön tesztek ellenőrzik, hogy a jelszó soha nem kerül vissza a response-ban, hogy a védett endpointok token nélkül 401-et adnak vissza, és hogy rossz jelszóval nem lehet bejelentkezni.

Teljesítmény ellenőrzések

Egyes tesztek a response idejét is mérik (pl. login alatt 2000ms, users lista alatt 1000ms, edzések listája alatt 1500ms).

Futtatási Sorrend

A collection-öket az alábbi sorrendben kell futtatni, mivel az auth token az első lépésben keletkezik:

- 1. Authentication Tests (auth_token itt generálódik)
- 2. Members Tests (test_member_id itt generálódik)
- 3. Trainings Tests
- 4. Users Tests (new_user_id itt generálódik)

4. Authentication Tests

Collection: BMFVSE - Authentication Tests | Endpoint csoport: /api/auth

Teszt 1: Login - Sikeres bejelentkezés

Endpoint: POST /api/auth/login
Prioritás: Kritikus
Jogosultság: Nem szükséges

Érvényes email és jelszó kombinációval teszteli a bejelentkezést. A sikeres login után a JWT token és a user ID automatikusan mentésre kerül environment változóba, amelyet a többi teszt használni fog.

Request body

```
{
    "email": "admin@bmfvse.hu",
    "password": "admin1234"
}
```

Tesztek

```
pm.test("Statusz kod 200", function () {
```

```
    pm.response.to.have.status(200);
});

pm.test("Response ideje kevesebb mint 2000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});

pm.test("Response tartalmaz token-t", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('token');
    pm.expect(jsonData.token).to.be.a('string');
    pm.expect(jsonData.token.length).to.be.above(0);
});

pm.test("Response tartalmaz user objektumot", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('user');
    pm.expect(jsonData.user).to.have.property('id');
    pm.expect(jsonData.user).to.have.property('roles');
});

pm.test("Token mentese environment valtozoba", function () {
    var jsonData = pm.response.json();
    pm.environment.set("auth_token", jsonData.token);
    pm.environment.set("user_id", jsonData.user.id);
});
```

PASS - Státusz 200, token visszakapva és elmentve, user objektum helyes struktúrával

Teszt 2: Login - Rossz jelszó

Endpoint: POST /api/auth/login
Prioritás: Kritikus
Jogosultság: Nem szükséges

Helytelen jelszóval próbál bejelentkezni. Ellenőrzi, hogy az API megfelelően kezeli a hibás hitelesítési adatokat, 401-es státuszkóddal válaszol és nem ad vissza tokent.

Request body

```
{
    "email": "admin@bmfvse.hu",
    "password": "wrongpassword"
}
```

Tesztek

```
pm.test("Statusz kod 401 - Unauthorized", function () {
    pm.response.to.have.status(401);
});

pm.test("Invalid credentials hiba", function () {
```

```
    var jsonData = pm.response.json();
    pm.expect(jsonData.error.message).to.equal('Invalid credentials');
});

pm.test("Nem kap token-t", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.not.have.property('token');
});
```

PASS - Státusz 401, 'Invalid credentials' hibaüzenet, token nem szerepel a válaszban

Teszt 3: Register - Sikeres regisztráció

Endpoint: POST /api/auth/register
Prioritás: Magas
Jogosultság: Nem szükséges

Új felhasználó regisztrációját teszteli egyedi, timestamp alapú email címmel. A pre-request script automatikusan generál egy egyedi email címet, hogy elkerülje az ismétlődő regisztrációt.

Pre-request script

```
const timestamp = Date.now();
pm.environment.set('random_email', `testuser${timestamp}@example.com`);
```

Request body

```
{
    "name": "Test User",
    "email": "{{random_email}}",
    "password": "password123",
    "phone": "+36301234567"
}
```

Tesztek

```
pm.test("Statusz kod 201 - Created", function () {
    pm.response.to.have.status(201);
});

pm.test("User sikeresen letrehozva", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.message).to.equal('User created successfully');
});

pm.test("Response tartalmaz user objektumot", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('user');
    pm.expect(jsonData.user).to.have.property('id');
});
```

PASS - Státusz 201, 'User created successfully' üzenet, user objektum ID-val visszaadva

Teszt 4: Get Current User - Auth token-nel

Endpoint: GET /api/auth/me
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

A bejelentkezett felhasználó adatait kéri le az auth token segítségével. Ellenőrzi a user objektum struktúráját és a roles tömb meglétét.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("User adatok helyes struktura", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('name');
    pm.expect(jsonData).to.have.property('roles');
});

pm.test("Roles tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.roles).to.be.an('array');
});
```

PASS - Státusz 200, user adatok id/name/roles mezőkkel, roles tömb típusú

5. Members Tests

Collection: BMFVSE - Members Tests | Endpoint csoport: /api/members

Teszt 5: Get All Members

Endpoint: GET /api/members
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

Az összes tag listáját kéri le. Ellenőrzi, hogy a válasz tömb típusú és a tagok tartalmazzák a kötelező mezőket: id, email, first_name, last_name.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
```

```
});

pm.test("Response tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('array');
});

pm.test("Members tartalmaznak kotelezo mezőket", function () {
    var jsonData = pm.response.json();
    if (jsonData.length > 0) {
        pm.expect(jsonData[0]).to.have.property('id');
        pm.expect(jsonData[0]).to.have.property('email');
        pm.expect(jsonData[0]).to.have.property('first_name');
        pm.expect(jsonData[0]).to.have.property('last_name');
    }
});
```

PASS - Státusz 200, tömb válasz, tagok id/email/first_name/last_name mezőkkel

Teszt 6: Get Member by ID

Endpoint: GET /api/members/1
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

Egy konkrét tag részletes adatait kéri le ID alapján. Ellenőrzi az objektum struktúráját és a tags property meglétét.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Member objektum helyes struktura", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('object');
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('email');
    pm.expect(jsonData).to.have.property('first_name');
    pm.expect(jsonData).to.have.property('last_name');
});

pm.test("Tags property letezik", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('tags');
});
```

PASS - Státusz 200, tag objektum helyes struktúrával, tags property megtalálható

Teszt 7: Create Member - Admin

Endpoint: POST /api/members

Prioritás: Kritikus
Jogosultság: Bearer {{auth_token}} (admin)

Új tag létrehozását teszteli admin jogosultsággal. A létrehozott tag ID-ját elmenti environment változóba, amelyet az Update és Delete tesztek használnak.

Request body

```
{
    "email": "testmember@bmfvse.hu",
    "password": "member123",
    "first_name": "Test",
    "last_name": "Member",
    "phone": "+36301111111"
}
```

Tesztek

```
pm.test("Statusz kod 201 - Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Uj member letrehozva", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('email');
    pm.expect(jsonData).to.have.property('first_name');
    pm.expect(jsonData).to.have.property('last_name');
});

pm.test("Member ID mentese", function () {
    var jsonData = pm.response.json();
    pm.environment.set("test_member_id", jsonData.id);
});
```

PASS - Státusz 201, új tag adatai visszaadva, test_member_id elmentve

Teszt 8: Update Member - Admin

Endpoint: PUT /api/members/{{test_member_id}}
Prioritás: Magas
Jogosultság: Bearer {{auth_token}} (admin)

A Create Member tesztben létrehozott tag adatait frissíti. Ellenőrzi, hogy a módosított adatok visszakerülnek a válaszban.

Request body

```
{
    "first_name": "Updated",
    "last_name": "Member",
```

```
    "email": "testmember@bmfvse.hu",
    "phone": "+36302222222"
}
```

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Member frissitve", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('phone');
});
```

PASS - Státusz 200, frissített tag adatai id és phone mezőkkel visszaadva

Teszt 9: Delete Member - Admin

Endpoint: DELETE /api/members/{{test_member_id}}
Prioritás: Magas
Jogosultság: Bearer {{auth_token}} (admin)

A korábban létrehozott teszt tagot törli. Ellenőrzi a sikeres törlés visszajelzését a válasz message mezőjén keresztül.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Sikeres torles", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('message');
    pm.expect(jsonData.message).to.include('deleted');
});
```

PASS - Státusz 200, 'deleted' szöveget tartalmazó message visszaadva

6. Trainings Tests

Collection: BMFVSE - Trainings Tests | Endpoint csoport: /api/trainings

Teszt 10: Get All Trainings

Endpoint: GET /api/trainings
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

Az összes jövőbeli edzés listáját kéri le. Ellenőrzi a struktúrát, az event_type mezőt (értéke 'training' kell legyen) és a response idejét (max 1500ms).

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('array');
});

pm.test("Training objektumok helyes struktura", function () {
    var jsonData = pm.response.json();
    if (jsonData.length > 0) {
        pm.expect(jsonData[0]).to.have.property('id');
        pm.expect(jsonData[0]).to.have.property('title');
        pm.expect(jsonData[0]).to.have.property('event_date');
        pm.expect(jsonData[0].event_type).to.equal('training');
    }
});

pm.test("Response ideje kevesebb mint 1500ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(1500);
});
```

PASS - Státusz 200, tömb válasz, training objektumok helyes struktúrával, teljesítmény OK

Teszt 11: Get Coaches List

Endpoint: GET /api/trainings/coaches
Prioritás: Közepes
Jogosultság: Bearer {{auth_token}}

Az edzők listáját kéri le. Ellenőrzi, hogy a válasz tömb típusú és az edző objektumok tartalmazzák az id, name és email mezőket.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Coaches lista tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('array');
});
```

```
pm.test("Coach objektumok helyes mezőket tartalmaznak", function () {
    var jsonData = pm.response.json();
    if (jsonData.length > 0) {
        pm.expect(jsonData[0]).to.have.property('id');
        pm.expect(jsonData[0]).to.have.property('name');
        pm.expect(jsonData[0]).to.have.property('email');
    }
});
```

PASS - Státusz 200, edzők listája id/name/email mezőkkel

Teszt 12: Get Training by ID

Endpoint: GET /api/trainings/1
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

Egy konkrét edzés részletes adatait kéri le ID alapján. Ellenőrzi a kötelező mezőket (id, title, event_date, location) és a creator_name property meglétét.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Training objektum helyes", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('object');
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('title');
    pm.expect(jsonData).to.have.property('event_date');
    pm.expect(jsonData).to.have.property('location');
});

pm.test("Creator name property letezik", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('creator_name');
});
```

PASS - Státusz 200, edzés részletes adatai id/title/event_date/location/creator_name mezőkkel

Teszt 13: Get Training Log - Admin

Endpoint: GET /api/trainings/log
Prioritás: Közepes
Jogosultság: Bearer {{auth_token}} (admin)

Az edzésnapló lekérése admin jogosultsággal - lejárt edzések részletekkel. Ellenőrzi a participants_count és creator_name mezők meglétét a log bejegyzésekben.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Training log tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('array');
});

pm.test("Log bejegyzesek tartalmaznak participant count-ot", function () {
    var jsonData = pm.response.json();
    if (jsonData.length > 0) {
        pm.expect(jsonData[0]).to.have.property('participants_count');
        pm.expect(jsonData[0]).to.have.property('creator_name');
    }
});
```

PASS - Státusz 200, edzésnapló tömb participants_count és creator_name mezőkkel

Teszt 14: Get Training without Auth - 401

Endpoint: GET /api/trainings
Prioritás: Kritikus
Jogosultság: Nincs (szándékosan)

Token nélkül próbál hozzáférni a védett edzés endpoint-hoz. Ellenőrzi, hogy az API megfelelően védi az erőforrásokat és 401-es státuszkóddal utasítja vissza a kérést.

Tesztek

```
pm.test("Statusz kod 401 - Unauthorized", function () {
    pm.response.to.have.status(401);
});

pm.test("Authentikacio szukseges hiba", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('error');
});
```

PASS - Státusz 401, error property a válaszban - védett endpoint megfelelően működik

7. Users Tests

Collection: BMFVSE - Users Tests | Endpoint csoport: /api/users

Teszt 15: Get All Users - Admin

Endpoint: GET /api/users
Prioritás: Magas
Jogosultság: Bearer {{auth_token}} (admin)

Az összes felhasználó listáját kéri le admin jogosultsággal. Fontos biztonsági ellenőrzés: a jelszó mező nem szerepelhet a válaszban. A response ideje 1000ms alatt kell legyen.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response tomb", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('array');
});

pm.test("Users tartalmaznak kotelezo mezőket", function () {
    var jsonData = pm.response.json();
    if (jsonData.length > 0) {
        pm.expect(jsonData[0]).to.have.property('id');
        pm.expect(jsonData[0]).to.have.property('email');
        pm.expect(jsonData[0]).to.not.have.property('password');
    }
});

pm.test("Response ideje kevesebb mint 1000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

PASS - Státusz 200, felhasználók listája jelszó nélkül, teljesítmény OK

Teszt 16: Get My Profile

Endpoint: GET /api/users/me
Prioritás: Magas
Jogosultság: Bearer {{auth_token}}

A bejelentkezett felhasználó saját profilját kéri le. Ellenőrzi a user objektum struktúráját, a roles property meglétét és azt, hogy a jelszó nem szerepel a válaszban.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("User objektum helyes struktura", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('object');
```

```
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('email');
    pm.expect(jsonData).to.not.have.property('password');
});

pm.test("Roles property letezik", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('roles');
});
```

PASS - Státusz 200, profil adatok roles mezővel, jelszó nem szerepel a válaszban

Teszt 17: Update My Profile

Endpoint: PATCH /api/users/me
Prioritás: Közepes
Jogosultság: Bearer {{auth_token}}

A bejelentkezett felhasználó saját profilját frissíti (telefon és cím). Ellenőrzi a módosított adatok visszaküldését és a biztonsági szabályt: jelszó nem szerepelhet a válaszban.

Request body

```
{
    "phone": "+36301234567",
    "address": "Budapest, Test utca 1."
}
```

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Modositott adatok visszajonnek", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('phone');
});

pm.test("Jelszo nincs a response-ban", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.not.have.property('password');
});
```

PASS - Státusz 200, módosított adatok visszaadva, jelszó nem szerepel a válaszban

Teszt 18: Create User - Admin

Endpoint: POST /api/users
Prioritás: Kritikus
Jogosultság: Bearer {{auth_token}} (admin)

Új felhasználó létrehozása admin jogosultsággal. Az új user ID-ját elmenti environment változóba a Delete teszt számára.

Request body

```
{
    "name": "New Test User",
    "email": "newuser@test.com",
    "password": "testpass123",
    "phone": "+36309876543"
}
```

Tesztek

```
pm.test("Statusz kod 201 - Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Uj user letrehozva", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('email');
});

pm.test("User ID mentese", function () {
    var jsonData = pm.response.json();
    pm.environment.set("new_user_id", jsonData.id);
});
```

PASS - Státusz 201, új user adatai visszaadva, new_user_id elmentve

Teszt 19: Delete User - Admin

Endpoint: DELETE /api/users/{{new_user_id}}
Prioritás: Magas
Jogosultság: Bearer {{auth_token}} (admin)

A Create User tesztben létrehozott felhasználót törli. Ellenőrzi a sikeres törlés visszajelzését.

Tesztek

```
pm.test("Statusz kod 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Sikeres torles uzenet", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('message');
    pm.expect(jsonData.message).to.include('deleted');
});
```

PASS - Státusz 200, 'deleted' szöveget tartalmazó message visszaadva

8. Futtatási Útmutató

Postman Collection Runner

- Nyisd meg a Postman alkalmazást
- Importáld mind a 4 collection JSON fájlt (File > Import)
- Hozz létre egy új Environment-et: bmfvse_local névvel
- Add hozzá a base_url változót: http://localhost:5000
- Válaszd ki az environment-et a jobb felső sarokban
- Kattints a collection neve melletti hárompontos menüre (...)
- Válaszd a 'Run collection' opciót
- Futtasd a collection-öket sorrendben: Auth > Members > Trainings > Users

Newman parancssori futtatás

A tesztek Newman segítségével parancssorból is futtathatók:

```
npm install -g newman
newman run authentication_tests.json --environment bmfvse_env.json
newman run members_tests.json --environment bmfvse_env.json
newman run trainings_tests.json --environment bmfvse_env.json
newman run users_tests.json --environment bmfvse_env.json
```

9. HTTP Státuszkódok Összefoglalója

Státuszkód	Jelentés	Mikor fordul elő
200 OK	Sikeres lekérés vagy frissítés	GET, PUT, PATCH, DELETE műveletek
201 Created	Sikeres létrehozás	POST műveletek (register, create)
401 Unauthorized	Hitelesítés szükséges	Hiányzó vagy érvénytelen token, rossz jelszó
404 Not Found	Az erőforrás nem található	Nem létező ID lekérése